Lab: Market Return Dynamics

We will repeat the weather analysis using the stock market's daily percentage change, recorded in `market_return_pct` in `ff_market_daily.parquet`.

- **2** means the market gained 2% that day.
- **−2** means the market lost 2%.
- **0** means no change.

Use `market_return_pct` as it is. Each row is one trading day, so the next row is the next day the market was open.

Does today's category help predict the next trading day's category?

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

df = pd.read_parquet('ff_market_daily.parquet')
df['date'] = pd.to_datetime(df['date'])
df = df.sort_values('date').reset_index(drop=True)

print(f"{len(df):,} trading-day observations: "
      f"{df['date'].min():%Y-%m-%d} to {df['date'].max():%Y-%m-%d}")
print(f"Are dates ordered? {df['date'].is_monotonic_increasing}")
df[['date', 'market_return_pct']].head()
```

```
26,296 trading-day observations: 1926-07-01 to 2026-07-31
Are dates ordered? True
```

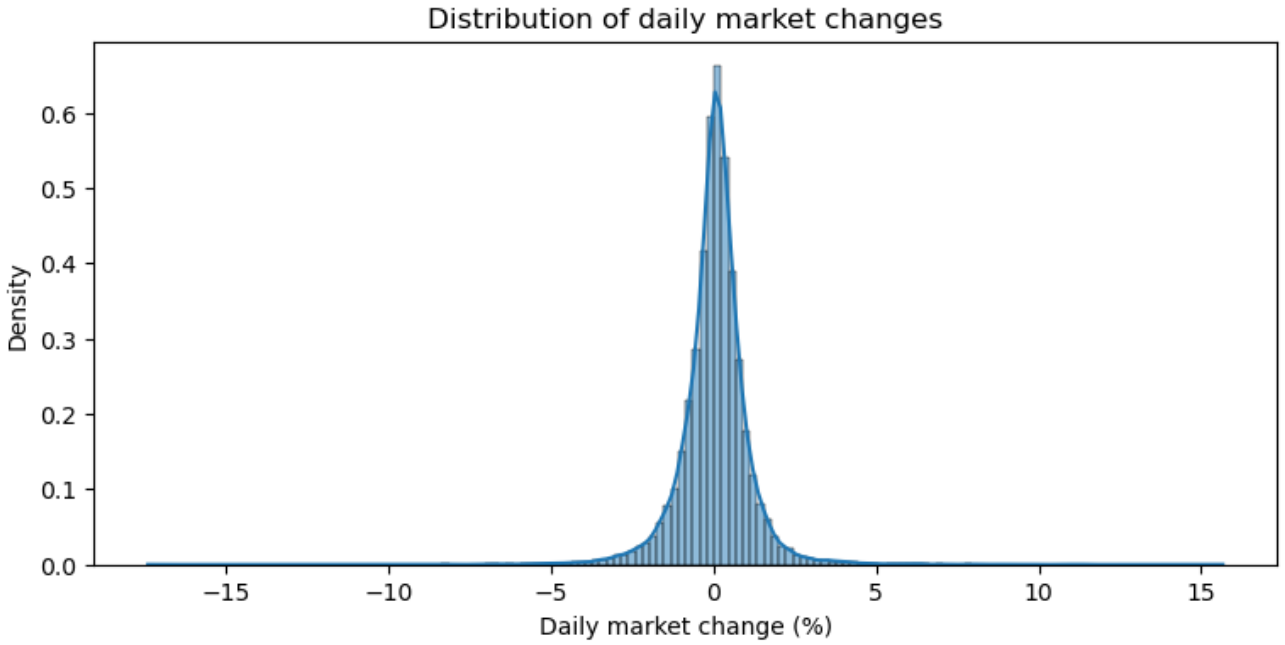
Out[3]:

	date	market_return_pct
0	1926-07-01	0.10
1	1926-07-02	0.46
2	1926-07-06	0.18
3	1926-07-07	0.10
4	1926-07-08	0.23

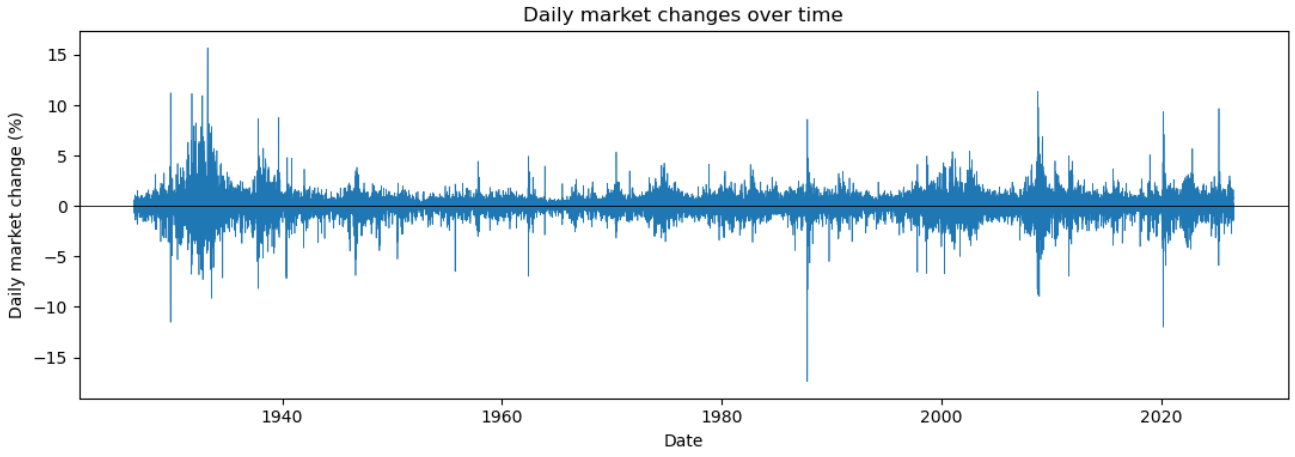
Here are plots of `market_return_pct`, as a kernel density plot and a time series.

In [4]:
```
fig, ax = plt.subplots(figsize=(9, 4))
```

```
sns.histplot(data=df, x='market_return_pct', bins=150, stat='density', kde=T
ax.set(xlabel='Daily market change (%)', title='Distribution of daily market
plt.show()
```



In [5]:
```
fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(df['date'], df['market_return_pct'], linewidth=0.5)
ax.axhline(0, color='black', linewidth=0.6)
ax.set(xlabel='Date', ylabel='Daily market change (%)',
       title='Daily market changes over time')
fig.tight_layout()
plt.show()
```



1. Two States

- Start with two categories: **Up** for values greater than 0, and **Down (and flat)** for values less than or equal to 0. Create variables in your dataframe to capture the

current and next value of `market_return_pct`.

- Pair today's category with the next trading day's category. Compute the joint PMF and marginals, and the conditional PMF.
- Are up transitions more or less common than down transitions?
- Are you more likely to transition to an Up day from a Down or Up day?

In [10]:
```
df['market_return_pct_today'] = df['market_return_pct']
df['market_return_pct_next'] = df['market_return_pct'].shift(-1)

state_order = ['Up', 'Down (and flat)']
df['state_today'] = np.where(df['market_return_pct_today'] > 0, 'Up', 'Down
df['state_next'] = np.where(df['market_return_pct_next'] > 0, 'Up', 'Down (a

transitions = df.dropna(subset=['market_return_pct_today', 'market_return_pc
joint_counts = pd.crosstab(transitions['state_today'], transitions['state_ne
joint_counts = joint_counts.reindex(index=state_order, columns=state_order,
joint_pmf = joint_counts / joint_counts.to_numpy().sum()

marginal_today = joint_pmf.sum(axis=1).rename('P(today)')
marginal_next = joint_pmf.sum(axis=0).rename('P(next)')
conditional_pmf = joint_pmf.div(marginal_today, axis=0)

print('Joint PMF:')
display(joint_pmf)
print('Marginal PMF for today:')
display(marginal_today.to_frame())
print('Marginal PMF for next trading day:')
display(marginal_next.to_frame())
print("Conditional PMF: next-day state given today's state:")
display(conditional_pmf)
```

```
Joint PMF:
```

state_next	Up	Down (and flat)
state_today		
Up	0.318996	0.228142
Down (and flat)	0.228142	0.224720

```
Marginal PMF for today:
```

	P(today)
state_today	
Up	0.547138
Down (and flat)	0.452862

```
Marginal PMF for next trading day:
```

state_next	P(next)
Up	0.547138
Down (and flat)	0.452862

```
Conditional PMF: next-day state given today's state:
```

state_today \ state_next	Up	Down (and flat)
Up	0.583026	0.416974
Down (and flat)	0.503779	0.496221

1. Up transitions are more common than down transitions, the stock market went up on 54.7 percent of the pairs in the data.
2. An Up day, given that today the stock went up, there is a 58.3 percent chance that it will go up the next day versus a 50.3 percent chance if the stock went down today.

2. Binning a Continuous Variable

Now keep the sign and a coarse measure of the size of a daily log return:

Category	Daily log return (%)
Large loss	$r \leq -1$
Small loss	$-1 < r \leq -0.1$
Near zero	$-0.1 < r \leq 0.1$
Small gain	$0.1 < r \leq 1$
Large gain	$r > 1$

For example, –2 belongs to **Large loss**, 0.05 to **Near zero**, and 0.8 to **Small gain**. These cutoffs are choices for this exercise. Each interval includes its upper endpoint; `-np.inf` and `np.inf` let the outer intervals include any size loss or gain.

- `pd.cut` to group `market_return_pct` into the five categories above. How many days fall into each of the five bins? Which are the most or least common?
- Compute the joint PMF and marginals. Which transitions are most likely to occur?

- Compute the conditional PMF. For each current state, which two future states are the most likely? Subjectively, how close is each row to a uniform mass function with probability .2 over each transition? Is it relatively easy or difficult to predict the next state?
- For each state, compute the conditional expectation of tomorrow's `market_return_pct`. Given the current state, what are the expected percent returns for tomorrow? How do the min and max values for tomorrow vary across states? Which states have the highest standard deviation for tomorrow's percent returns? Describe any patterns in the volatility you notice. Using `.groupby` and `.describe` is the easiest way to do this.

In [ ]:
```
bins = [-np.inf, -1, -0.1, 0.1, 1, np.inf]
labels = ['Large loss', 'Small loss', 'Near zero', 'Small gain', 'Large gair
df['b_t'] = pd.cut(df['r_t'], bins=bins, labels=labels, right=True)
df['b_tp1'] = pd.cut(df['r_tp1'], bins=bins, labels=labels, right=True)


counts = df['b_t'].value_counts().reindex(labels)
print('Days per bin:\n', counts, '\n')

pairs5 = df.dropna(subset=['r_tp1'])


joint5 = pd.crosstab(pairs5['b_t'], pairs5['b_tp1'], normalize='all')
joint5.index.name, joint5.columns.name = 'Today', 'Tomorrow'
print('Joint PMF:\n', joint5.round(4), '\n')
print('Marginal (today):\n', joint5.sum(axis=1).round(4), '\n')
print('Marginal (tomorrow):\n', joint5.sum(axis=0).round(4), '\n')
print('Most likely transitions:\n', joint5.stack().sort_values(ascending=Fal

cond5 = joint5.div(joint5.sum(axis=1), axis=0)
print('Conditional PMF (rows = today):\n', cond5.round(4), '\n')
for s in labels:
    top2 = cond5.loc[s].nlargest(2)
    print(f'{s:>10}: ' + ', '.join(f'{k} ({v:.3f})' for k, v in top2.items()
print()


print(pairs5.groupby('b_t', observed=True)['r_tp1'].describe().round(3))
```

```
Days per bin:
 b_t
Large loss    2759
Small loss    7452
Near zero     3441
Small gain    9785
Large gain    2859
```

```
Name: count, dtype: int64

Joint PMF:
 Tomorrow    Large loss  Small loss  Near zero  Small gain  Large gain
Today
Large loss      0.0213      0.0272     0.0091      0.0278      0.0195
Small loss      0.0354      0.0886     0.0342      0.0978      0.0274
Near zero       0.0093      0.0420     0.0210      0.0493      0.0093
Small gain      0.0250      0.1001     0.0550      0.1594      0.0326
Large gain      0.0138      0.0255     0.0116      0.0378      0.0199

Marginal (today):
 Today
Large loss    0.1049
Small loss    0.2834
Near zero     0.1309
Small gain    0.3721
Large gain    0.1087
dtype: float64

Marginal (tomorrow):
 Tomorrow
Large loss    0.1049
Small loss    0.2834
Near zero     0.1308
Small gain    0.3721
Large gain    0.1087
dtype: float64

Most likely transitions:
 Today       Tomorrow
Small gain  Small gain    0.1594
            Small loss    0.1001
Small loss  Small gain    0.0978
            Small loss    0.0886
Small gain  Near zero     0.0550
dtype: float64

Conditional PMF (rows = today):
 Tomorrow    Large loss  Small loss  Near zero  Small gain  Large gain
Today
Large loss      0.2033      0.2592     0.0863      0.2650      0.1863
Small loss      0.1251      0.3128     0.1205      0.3450      0.0966
Near zero       0.0712      0.3208     0.1601      0.3769      0.0709
Small gain      0.0672      0.2689     0.1479      0.4285      0.0876
Large gain      0.1273      0.2347     0.1070      0.3477      0.1833

Large loss: Small gain (0.265), Small loss (0.259)
Small loss: Small gain (0.345), Small loss (0.313)
 Near zero: Small gain (0.377), Small loss (0.321)
```

```
Small gain: Small gain (0.428), Small loss (0.269)
Large gain: Small gain (0.348), Small loss (0.235)

             count   mean    std    min   25%   50%   75%    max
b_t
Large loss  2759.0 -0.038  1.721 -17.41 -0.79 -0.01  0.72  11.22
Small loss  7452.0 -0.044  1.010 -11.23 -0.53  0.01  0.45  11.36
Near zero   3441.0  0.015  0.826  -8.19 -0.35  0.03  0.42   5.49
Small gain  9784.0  0.107  0.832  -8.25 -0.25  0.12  0.50   8.67
Large gain  2859.0  0.163  1.384 -12.00 -0.37  0.15  0.73  15.67
```

1. Large Loss: 2,759 days Small Loss:7,452 days Near Zero: 3,441 days, Small Gain: 9,785 days

Large Gain: 2,859 days. Small Gain is the most common and Large Loss is the least common.

2. Small gain to Small gain is most likely to occur with a probability of 15.94 percent.

After that it is small gain to small loss and small loss to small gain.

3. Small loss and small gain are the most common conditional on all five states. None of them are particularly close to a uniform distribution. Large loss is the closest to uniform, Near zero is the only state that is over 0.1 from uniform (0.2). Near zero is the farthest from uniform with small four of the five states being over 0.1 away from 0.2. It is still difficult to predict the next state even though the distribution is not uniform. A lot of the variance is based on the natural occurrence of each state.

4. Expected return tomorrow given today's state:

Large loss: −0.04% Small loss: −0.04% Near zero: +0.02% Small gain: +0.11% Large gain: +0.16%

These rise from losses to gains, which is slight momentum, but they're all tiny compared with daily swings of about 1%.

Min and max tomorrow:

Large loss: −17.4 to 11.2 Small loss: −11.2 to 11.4 Near zero: −8.2 to 5.5 Small gain: −8.3 to 8.7 Large gain: −12.0 to 15.7

The range is widest after the large states, in both directions, and narrowest after calm

days. The single worst day and the single best day both follow large moves.

Standard deviation tomorrow:

Large loss: 1.72 (highest) Large gain: 1.38 Small loss: 1.01 Small gain: 0.83 Near zero: 0.83

The volatility pattern is U-shaped: big moves today, up or down, are followed by more volatile days, and calm days are followed by calm days. That's volatility clustering. It's also asymmetric: a large loss raises tomorrow's volatility more than a large gain, and a small loss more than a small gain. So the direction of tomorrow's move is close to unpredictable, but how big it's likely to be is somewhat predictable.

3. Scaling up

The choice of five bins was arbitrary. For more realistic or complex models, we'd want to flexibly choose the number of bins. Instead of the five bins from above, we'll break the data into 21 equally spaced bins based on quantile; the qualitative results hold up for 11 or 71 or 101, but 21 is interpretable and easier to manage.

- Use `pd.qcut` to assign the data into 21 bins, and add that variable to the dataframe as `df[y_t]`. Use `.shift(-1)` to create `df[y_tp1]`.
- Compute the joint PMF. Plot it as a heatmap. Do you see any obvious patterns? Explain. Try subtracting 1/21**2 from the PMF before plotting it, to normalize away the uniform PMF, and see systematic deviations from uniformly random behavior.
- Compute the conditional PMF. Plot it as a heatmap. Do you see any obvious patterns? Explain. Try subtracting 1/21 from the PMF before plotting it, to normalize away the uniform PMF, and see systematic deviations from uniformly random transitions.
- For each bin, compute the next day's minimum, mean, and maximum values. Plots these as lines on a graph against the bins. Do you see any obvious patterns? Explain.
- For each bin, compute the next day's standard deviation, and plot it as a line graph against the bins. Do you see any obvious patterns? Explain.
- How easy is it to predict the market?

In [ ]:
```
K = 21

df['y_t'] = pd.qcut(df['market_return_pct'], q=K, labels=False)
df['y_tp1'] = df['y_t'].shift(-1)
print('Days per bin:', df['y_t'].value_counts().sort_index().tolist())
```

```
pairs21 = df.dropna(subset=['y_tp1']).astype({'y_tp1': int})


joint21 = pd.crosstab(pairs21['y_t'], pairs21['y_tp1'], normalize='all')
joint21.index.name, joint21.columns.name = 'Today (bin)', 'Tomorrow (bin)'

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))
sns.heatmap(joint21, cmap='viridis', square=True, ax=axes[0])
axes[0].set_title('Joint PMF')
dev = joint21 - 1 / K**2
lim = np.abs(dev.values).max()
sns.heatmap(dev, cmap='RdBu_r', center=0, vmin=-lim, vmax=lim, square=True,
axes[1].set_title('Joint PMF minus uniform (1/21²)')
for ax in axes:
    ax.invert_yaxis()
fig.tight_layout()
plt.show()

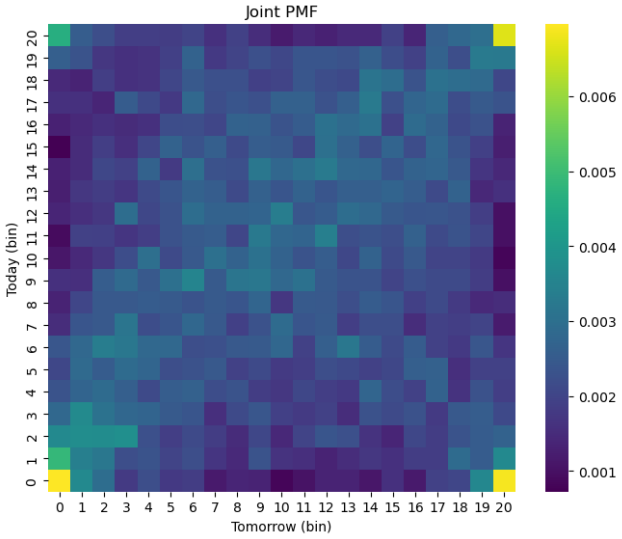

cond21 = joint21.div(joint21.sum(axis=1), axis=0)

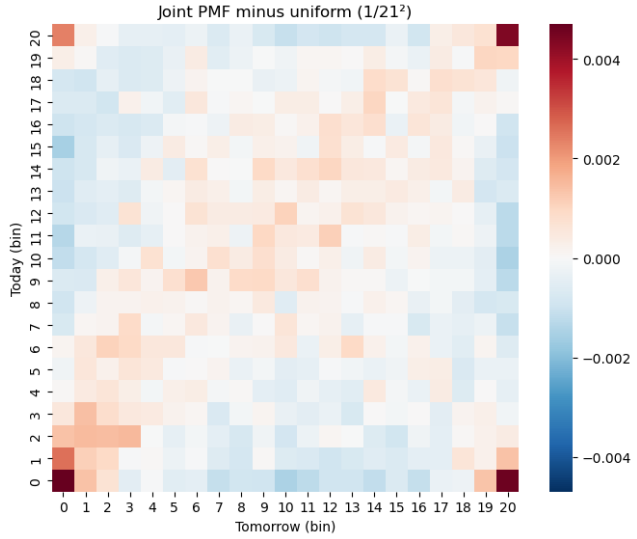
fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))
sns.heatmap(cond21, cmap='viridis', square=True, ax=axes[0])
axes[0].set_title('Conditional PMF  P(tomorrow | today)')
dev = cond21 - 1 / K
lim = np.abs(dev.values).max()
sns.heatmap(dev, cmap='RdBu_r', center=0, vmin=-lim, vmax=lim, square=True,
axes[1].set_title('Conditional PMF minus uniform (1/21)')
for ax in axes:
    ax.invert_yaxis()
fig.tight_layout()
plt.show()

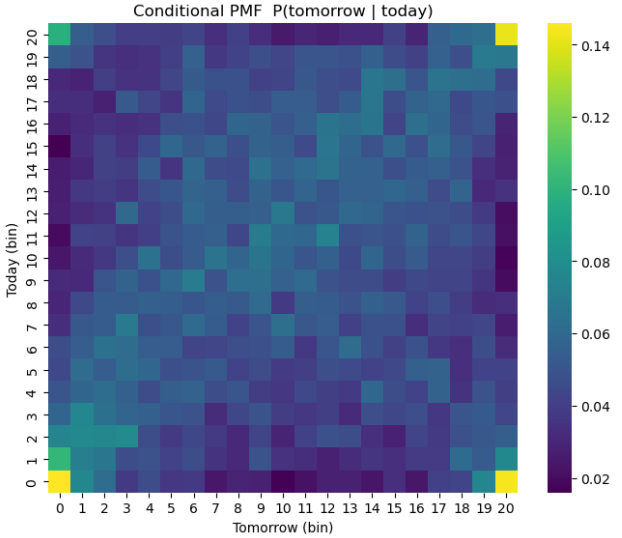
nxt = pairs21.groupby('y_t')['r_tp1'].agg(['min', 'mean', 'max', 'std'])
print(nxt.round(3))

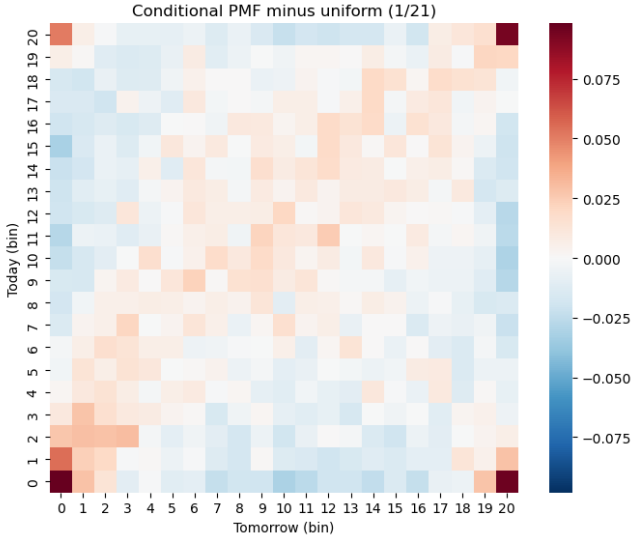
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
for col in ['min', 'mean', 'max']:
    axes[0].plot(nxt.index, nxt[col], marker='o', label=col)
axes[0].axhline(0, color='black', linewidth=0.6)
axes[0].set(xlabel="Today's bin", ylabel="Next day's return (%)",
            title="Next day's min, mean, max by today's bin", xticks=range(K
axes[0].legend()
axes[1].plot(nxt.index, nxt['std'], marker='o')
axes[1].set(xlabel="Today's bin", ylabel="Std of next day's return (%)",
            title="Next day's standard deviation by today's bin", xticks=rar
fig.tight_layout()
plt.show()
```

```
Days per bin: [1255, 1253, 1286, 1280, 1230, 1226, 1371, 1206, 1198, 1331, 1
210, 1223, 1294, 1223, 1310, 1193, 1211, 1283, 1225, 1250, 1238]
```

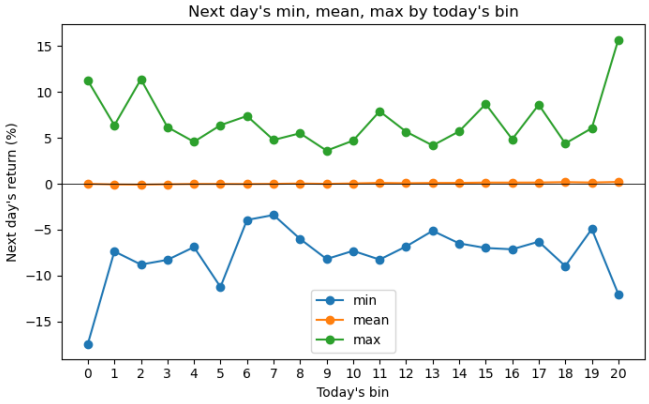


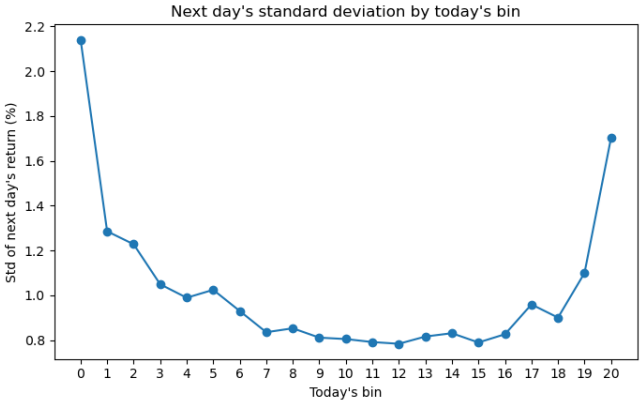






y_t	min	mean	max	std
0	−17.41	−0.014	11.22	2.140
1	−7.36	−0.077	6.36	1.286
2	−8.79	−0.097	11.36	1.228
3	−8.28	−0.059	6.16	1.050
4	−6.89	−0.024	4.56	0.989
5	−11.23	−0.023	6.37	1.025
6	−3.94	−0.030	7.37	0.930
7	−3.40	−0.014	4.76	0.836
8	−6.00	0.026	5.49	0.853
9	−8.19	−0.013	3.60	0.812
10	−7.30	0.035	4.69	0.805
11	−8.25	0.092	7.88	0.792
12	−6.82	0.057	5.66	0.784
13	−5.12	0.088	4.16	0.817
14	−6.49	0.091	5.70	0.831
15	−6.99	0.126	8.67	0.790
16	−7.13	0.126	4.84	0.827
17	−6.30	0.130	8.60	0.960
18	−8.96	0.175	4.38	0.901
19	−4.93	0.133	6.04	1.100
20	−12.00	0.195	15.67	1.703





1. Binning: qcut puts about 1,250 days in each of the 21 bins. The bins have equal counts but very different widths. The extreme bins are wide, for example bin 0 runs from –17.4% to –1.6%, and the middle bins are narrow, for example bin 10 runs from 0.04% to 0.11%.

2. Joint PMF: it's nearly uniform at about 1/441 per cell. After subtracting 1/441, the four corners stand out, so extreme days tend to be followed by extreme days in either direction. Calm days tend to follow calm days.

3. Conditional PMF: same pattern. After an extreme day, the chance tomorrow is extreme is about 0.14 versus 0.048 under uniform, which is roughly three times

higher. The other cells are close to 1/21.

4. Min, mean and max: the mean stays close to zero for every bin. The min and max are widest after extreme days, so a big day is followed by a bigger range in both directions.

5. Standard deviation: U-shaped. It's about 2.1 after the lowest bin, 0.8 in the middle and 1.7 after the highest bin. That's volatility clustering, and it's stronger after losses.

6. Predictability: the direction is almost impossible to predict, but how volatile tomorrow will be is fairly predictable.